THE OPEN SOURCE AUDIT

A Capstone Project Report

Open Source Software (OSS NGMC) | VIT Bhopal University

Software Chosen	Python — Programming Language
License	Python Software Foundation (PSF) License
Student Name	FAIZ KHAN
Reg. Number	24BCE10703
Slot	E22
Date of Submission	March 30, 2026

Table of Contents

1. **Introduction** 3

Part A **Origin and Philosophy**

- A1. The Problem Python Was Created to Solve 4
- A2. The License — What It Actually Says 5
- A3. The Ethics of Open Source 7

Part B **Linux Footprint** 9

Part C **The FOSS Ecosystem** 11

Part D **Open Source vs Proprietary — Critical Analysis** 12

Part E **Shell Script Documentation** 13

Conclusion 16

References 16

1. Introduction

This report presents a structured audit of **Python**, one of the most widely used open-source programming languages in the world today. The audit examines Python not simply as a technical tool, but as the product of deliberate philosophical choices — choices about who should own software, who should be able to read it, and what happens when knowledge is shared freely rather than locked behind a paywall.

Python was selected for this audit because it sits at a rare intersection: it is simultaneously a beginner's first language and a professional's production tool. It powers data science pipelines at Google and research experiments at universities. It runs automation scripts at startups and introductory programming courses at colleges across India. Understanding why Python is open-source, what its license actually permits, and how it lives on a Linux system is not abstract preparation — it is preparation for a career where Python will almost certainly appear in some capacity.

The report is structured as follows. Part A examines Python's origin story, its license, and the ethics of open-source software. Part B documents Python's physical presence on a Linux system. Part C maps the broader FOSS ecosystem Python belongs to. Part D compares Python honestly against its closest proprietary alternative. Part E documents the five shell scripts written as part of this project. The report closes with a conclusion and references.

Part A — Origin and Philosophy

A1. The Problem Python Was Created to Solve

It is easy to assume that Python was designed to be the language of data science or artificial intelligence. It was not. It was designed for something far more modest: to be a language that humans could actually read.

In the late 1980s, Guido van Rossum was working at Centrum Wiskunde & Informatica (CWI) in Amsterdam, contributing to a language called ABC. ABC was a teaching language — clean, structured, intentionally simple. It was not without merit. But it had one crippling limitation: it was completely closed. Developers could not extend it, could not adapt it to new platforms, and could not submit improvements. If ABC had a flaw, that flaw stayed. The source was not available to the people using it.

Simultaneously, van Rossum was frustrated with the scripting options available for the Amoeba distributed operating system. Shell scripting was too limited for any task of real complexity. C was expressive but demanded so much boilerplate that writing a quick utility script was an exercise in patience rather than problem-solving. There was a gap between 'trivial' and 'full program' that had no good occupant.

Over the Christmas holiday of 1989, van Rossum began filling that gap. He named the result after Monty Python's Flying Circus — a deliberate signal that the language was meant to be enjoyable, not imposing. Python 0.9.0 was released in 1991, and crucially, it was released with its full source code and an open license. Van Rossum could have kept it proprietary. He had built something genuinely useful. He chose not to.

The consequences of that choice have compounded over thirty years. Because Python was open, university departments could teach with it without licensing negotiations. Because it was open, developers could build NumPy, Pandas, and TensorFlow on top of it without seeking permission from a corporate owner. Because it was open, when van Rossum stepped back from his BDFL (Benevolent Dictator for Life) role in 2018, the Python Steering Council could take over governance without the language dying or being acquired. Every significant capability Python has today — its data science ecosystem, its web frameworks, its AI libraries — exists because an open foundation made large-scale collaboration possible.

Note: Python's name comes from *Monty Python's Flying Circus*, not the snake. Van Rossum wanted a name that was short, unique, and slightly irreverent — a deliberate contrast to the serious, acronym-heavy naming conventions of the era.

A2. The License — What It Actually Says

The Four Freedoms of Free Software

The Free Software Foundation defines four essential freedoms that any genuinely free software must guarantee to its users. These freedoms are not abstract ideals — each one has concrete implications for how software can be built, shared, and built upon.

- **Freedom 0 — The freedom to run**

  You may use Python for any purpose. There are no restrictions based on use case — commercial, academic, governmental, or personal. A startup and a university share the same rights to run Python.

- **Freedom 1 — The freedom to study and change**

  Python's complete source code is publicly accessible on GitHub and at python.org. Any developer can read the interpreter, understand how it works, and modify it.

- **Freedom 2 — The freedom to redistribute**

  You may share Python with others — freely or for a fee. There is no licence activation, no serial number, no restriction on who receives a copy.

- **Freedom 3 — The freedom to distribute modified versions**

  If you improve Python, you may release your modified version. Others can then benefit from your improvements.

The Python Software Foundation (PSF) License

Python is released under the **Python Software Foundation License (PSF License)**, a permissive open-source licence approved by both the Open Source Initiative (OSI) and the Free Software Foundation (FSF). The key characteristics of the PSF Licence are:

- A company may take Python's source code, modify it, and incorporate it into a proprietary product **without being legally required to publish their changes**. This distinguishes it sharply from the GPL.
- The original copyright notice and licence text must be retained in any distribution.
- The Python Software Foundation's name may not be used to endorse or promote products derived from Python without written permission.
- There is **no copyleft provision** — derived works do not have to remain open.

This permissiveness is deliberate. The Python community chose a licence designed to minimise friction for adoption. The reasoning: if companies can use Python freely without licence compatibility concerns, they will adopt it more widely; wider adoption creates more developers who know Python; more Python developers means a larger pool of potential contributors; and more contributors improves the language for everyone. It is an ideological trade-off — practical reach

over strict ideological purity.

Comparison: GPL vs MIT vs PSF

Dimension	GPL v2	MIT	PSF (Python)
Commercial use allowed?	Yes	Yes	Yes
Must publish modifications?	Yes (copyleft)	No	No
Can use in proprietary software?	Only if GPL-compatible	Yes	Yes
Attribution required?	Yes	Yes	Yes
Patent grant included?	No explicit clause	No	No
Licence length	Long, detailed	Short	Medium

If I were choosing a licence for a library intended for widest possible adoption — especially in academic and commercial settings — I would choose **MIT**. It is brief, universally understood, and creates no compatibility friction. If I were building infrastructure I specifically wanted to keep permanently open — where I did not want corporations to take the work, improve it privately, and compete with their closed version — I would choose **GPL v2**. The choice is not technical. It is a statement about what you value: reach or ideological consistency.

Free as in Beer vs Free as in Freedom

These two phrases describe completely different things that unfortunately use the same word. *"Free as in free beer"* means no monetary cost. *"Free as in freedom"* means you control what happens to the software.

Adobe Acrobat Reader is free as in beer — you download it at no cost, but you cannot read its source, cannot modify it, and cannot redistribute it. Python is both: it costs nothing **and** you have the four freedoms. The distinction matters because freedom persists even when the pricing changes. An open-source project that begins charging for distribution still gives you the code; a proprietary tool that starts charging removes both your money and your control.

A3. The Ethics of Open Source

Should All Software Be Open Source?

The case for universal open source is not without substance. Software has become the infrastructure of modern life — it runs hospitals, banks, schools, elections, and civil services. When infrastructure is opaque, it cannot be held accountable. A proprietary algorithm making loan decisions, a closed electoral system, a medical device whose code cannot be audited — these are not merely inconveniences. They are structural risks. Open source enforces a kind of transparency that proprietary software structurally cannot. Anyone can inspect it. Anyone can catch what is wrong.

The case against is equally serious, and less often acknowledged. Software development costs real money. The engineers who built the first Python interpreter, who wrote the NumPy library, who maintain the OpenSSL stack that secures a significant portion of the world's internet traffic — these people need to eat. Open-source sustainability is a genuine crisis, not a solved problem. The **Heartbleed vulnerability in 2014** — a critical flaw in OpenSSL — was partly a consequence of critical infrastructure being maintained by a two-person team with almost no funding. Making all software open-source without solving the compensation problem does not produce more good software; it produces more broken software maintained by burnt-out contributors who eventually stop.

The honest conclusion is that open source is not a universal solution. It is the right model for infrastructure, standards, and developer tooling — and a fragile model for anything that requires long-term funded maintenance without an attached business model. Python works because the Python Software Foundation, corporate sponsors, and employed contributors provide the necessary resources. Not every project has that.

Is It Ethical to Build Profitable Products on Free Community Labour?

This is the question that companies like Google, Meta, and Amazon prefer not to answer directly. These organisations have built multi-billion dollar businesses on top of Linux, Python, TensorFlow, and dozens of other projects maintained largely by volunteers or by developers at smaller organisations. The companies do contribute back — Google employs core Python developers, Meta released PyTorch, Amazon contributes to the Linux kernel — but contributions are selective and self-serving. They contribute what benefits them competitively and retain as proprietary whatever gives them advantage.

There is a recognised pattern for this behaviour: the **"extraction model"**. The community builds the foundation; the corporation extracts the value; the community receives attribution but not compensation. Whether this is unethical depends on what you think contributors agreed to when they chose a permissive licence. If a developer releases code under MIT, they explicitly permitted anyone to use it however they wished. Blaming a corporation for doing exactly what the licence says is permitted is genuinely difficult. But the systemic outcome — where the people doing the essential work are not the people capturing the economic value — is a structural problem that

permissive licences create by design, whether or not any individual actor is behaving improperly.

Standing on the Shoulders of Giants

Every piece of code I write as a student runs on layers I did not build: the Python interpreter, the Linux kernel, the GNU C compiler, the NumPy linear algebra routines, the BLAS libraries beneath NumPy. The people who built those layers stood on layers before them. Open source makes this possible in a way proprietary software structurally cannot — because the layers are visible, inspectable, and modifiable by anyone who needs to understand them.

"Standing on the shoulders of giants" in software does not mean passively using existing work. It means being able to look at what previous developers built, understand precisely how and why it works, and build something genuinely new on top of it. That requires access to the source. Proprietary foundations prevent that kind of cumulative knowledge-building. Open source enables it — which is why the most significant innovations in computing over the past thirty years have mostly come from the open-source world, not from closed, proprietary development.

Part B — Linux Footprint

Python is a native citizen of Linux. It ships pre-installed with almost every major Linux distribution, is embedded in package management tools written in Python itself, and is the default scripting language for system administration across the Linux world. Understanding how Python physically lives on a Linux system — where it resides, how it is managed, and what user it runs under — is essential for any developer working in a Linux environment.

Installation Method

On **Debian-based distributions** (Ubuntu, Kali Linux, Debian), Python 3 is installed via APT:

```
sudo apt update sudo apt install python3 python3-pip
```

On **RPM-based distributions** (Fedora, CentOS, RHEL):

```
sudo dnf install python3 python3-pip
```

Python may also be compiled from source when a version not available in the distribution's repository is required. This involves downloading the source tarball from python.org, then running the standard build sequence:

```
./configure --enable-optimizations make -j$(nproc) sudo make altinstall
```

The altinstall target is used instead of install to avoid overwriting the system Python that many Linux tools depend on.

User and Group

Python scripts run under whatever Linux user invokes them — there is no dedicated system user for Python itself, unlike services such as Apache HTTP Server (which runs under the www-data user for security isolation). This is both a strength and a risk.

The strength: Python is flexible and can be used in any user context without additional configuration. The risk: a Python script running as root has root-level access to the entire system.

Best practice for any Python process that runs persistently — a web application, a background worker, a scheduled job — is to create a dedicated, low-privilege user account for that process and to use Python virtual environments to isolate its dependencies from the system Python installation.

Service Management with systemd

Python itself is not a service — it is an interpreter. However, Python applications are routinely managed as persistent services using **systemd**. A unit file placed in /etc/systemd/system/myapp.service might look like this:

```
[Unit] Description=My Python Application After=network.target [Service]
ExecStart=/usr/bin/python3 /home/appuser/app.py User=appuser
WorkingDirectory=/home/appuser Restart=always RestartSec=5 [Install]
WantedBy=multi-user.target
```

Common service management commands:

```
sudo systemctl start myapp # Start the service sudo systemctl stop myapp # Stop the
service sudo systemctl restart myapp # Restart after changes sudo systemctl status myapp #
Check running status sudo systemctl enable myapp # Auto-start on boot
```

Security Patch and Update Model

Security patches reach Python on Linux systems through two distinct channels.

- **Distribution patches:** Distro maintainers (Ubuntu, Fedora, Debian security teams) backport security fixes into the specific Python version they ship. A system running Ubuntu 22.04 with Python 3.10 receives security patches for that version through sudo apt upgrade, even if upstream Python has moved to 3.12.
- **PyPI package patches:** Third-party packages installed via pip are updated independently. The Python Security Response Team coordinates responsible disclosure and patch releases for the CPython interpreter itself. Tools like pip audit and GitHub's Dependabot help developers identify vulnerable packages in their projects.

Part C — The FOSS Ecosystem

What Python Depends On

Python does not exist in isolation on a Linux system. Its CPython implementation is written primarily in C, and compiling it from source requires the **GCC (GNU Compiler Collection)** and **GNU Make** — both GPL-licensed tools. The standard library's ssl module links against **OpenSSL** for cryptographic operations. The zlib module links against the zlib compression library. SQLite support is provided by linking against the public-domain SQLite database engine. The tkinter GUI module wraps the Tcl/Tk toolkit.

Python is, in a concrete sense, assembled from other open-source components. This is not incidental — it is characteristic of how the FOSS ecosystem works. No project is built from scratch; every project stands on the foundations laid by others.

What Python Has Enabled and Inspired

The ecosystem that has grown around Python's open core is arguably more consequential than the language itself. Key developments include:

- **Scientific computing:** NumPy, SciPy, Matplotlib, and Pandas transformed scientific computing and enabled the data science revolution of the 2010s.
- **Machine learning:** TensorFlow (Google), PyTorch (Meta), and scikit-learn are all Python-first frameworks that would not exist in their current form without the Python ecosystem.
- **Web development:** Django and Flask power millions of websites. Gunicorn and uWSGI serve Python web applications behind Nginx and Apache.
- **DevOps and automation:** Ansible (IT automation), Fabric (deployment), and Salt (configuration management) are all written in Python.
- **Inspired languages:** Julia was created partly as a scientific-computing alternative to Python. Cython bridges Python and C for performance-critical code.

Python in the Web Stack

The classic LAMP stack (Linux, Apache, MySQL, PHP) predates Python's web prominence, but Python has its own equivalent — often called the **LLMP stack**: Linux, Nginx, Python (via WSGI/ASGI), and PostgreSQL or MySQL. Flask and Django are the dominant web frameworks. In the modern cloud era, Python-based APIs served by FastAPI or Django REST Framework behind an Nginx reverse proxy represent a standard architecture for web services.

Python interacts with the web stack's data layer through SQLAlchemy (an ORM supporting PostgreSQL, MySQL, and SQLite), psycopg2 (direct PostgreSQL driver), and PyMySQL. Every component in this chain — from the web framework to the database driver — is open source.

Community and Governance

Python's community is one of the most deliberately structured in open source. After van Rossum's retirement as BDFL in 2018, Python adopted a **five-member Steering Council** model, elected annually by active core developers. Major language changes are proposed and debated through a formal **Python Enhancement Proposal (PEP)** process — a written proposal, community discussion period, and Steering Council vote.

The **Python Software Foundation (PSF)** is a US 501(c)(3) non-profit that owns the CPython project, manages the python.org domain, organises PyCon (the annual conference), and distributes grants to regional community events. The PSF is funded through PyCon ticket sales, corporate sponsorships (from Google, Microsoft, Bloomberg, and others), and individual memberships.

Community spaces include the python-dev and python-ideas mailing lists, the discuss.python.org forum, the CPython GitHub repository, and regional conferences (PyCon India, EuroPython, PyCon Africa, PyCon APAC). Python is unusual among major open-source projects in having explicit diversity initiatives and a Code of Conduct that is actively enforced.

Part D — Open Source vs Proprietary: Critical Analysis

The closest proprietary alternative to Python as a general-purpose language for scientific computing, data analysis, and engineering automation is **MATLAB**, developed by MathWorks. MATLAB has been the dominant proprietary environment for numerical computing in academia and industry for over three decades. The comparison below attempts to be genuinely honest about both options — not to assume open source wins by default.

Overall Verdict and Recommendation

For a real-world deployment — a startup building a data pipeline, a researcher at a university with limited budget, a developer writing automation tooling — **Python is the pragmatic choice in almost every scenario**. The zero licensing cost alone is decisive at scale: a fifty-person data team using MATLAB at institutional pricing represents a significant recurring cost with no technical justification over Python for most workloads.

The situations where MATLAB remains the correct choice are specific: **(a)** where a MathWorks toolbox provides validated, certified algorithms required for regulatory compliance — aerospace simulations, medical device signal processing — where the vendor warranty on the algorithm's correctness is part of the product's safety case; or **(b)** where a team is deeply embedded in a legacy MATLAB workflow and migration costs genuinely outweigh the benefits.

Would I contribute to Python? Yes — not to the CPython core immediately, which requires significant expertise in C and language theory, but to the wider ecosystem. Improving documentation, filing well-described bug reports, and contributing to libraries like Requests or Pandas are realistic starting points. Open-source contribution begins with using the software seriously, noticing what is missing or broken, and taking the step of submitting a fix rather than merely working around the problem.

Part E — Shell Script Documentation

Five shell scripts were written as part of this project to demonstrate practical Linux command-line skills. Each script is presented below with its complete source code, a description of its function, and an explanation of the shell scripting concepts it demonstrates. All scripts have been tested on Ubuntu 22.04 LTS.

Script 1 — System Identity Report (system_identity.sh)

Description

This script generates a formatted welcome screen displaying system identity information: the Linux distribution name, kernel version, currently logged-in user, home directory path, system uptime, and current date and time. It closes with a note about the GPL licence that governs the Linux OS.

Source Code

```
#!/bin/bash # Script 1: System Identity Report # Author: FAIZ KHAN| Reg: 24BCE10703#
Course: Open Source Software | VIT Bhopal University # Variables — store student and
software details STUDENT_NAME="FAIZ KHAN" SOFTWARE_CHOICE="Python" # Command substitution
— each $(...) runs a command and captures its output KERNEL=$(uname -r) # Kernel version
USER_NAME=$(whoami) # Logged-in username HOME_DIR=$HOME # Home directory (built-in
variable) UPTIME=$(uptime -p) # Human-readable uptime CURRENT_DATE=$(date) # Full date and
time string # Get distro name; fall back gracefully if lsb_release is unavailable if
command -v lsb_release &>/dev/null; then DISTRO=$(lsb_release -d | cut -f2) else
DISTRO="Unknown distribution" fi # Display — formatted output using echo echo
"========================================================" echo " Open Source Software
Audit — $STUDENT_NAME" echo "========================================================"
echo " Software : $SOFTWARE_CHOICE" echo " Kernel : $KERNEL" echo " Distro : $DISTRO" echo
" User : $USER_NAME" echo " Home : $HOME_DIR" echo " Uptime : $UPTIME" echo " Date :
$CURRENT_DATE" echo " Licence : GPL-based Linux system" echo
"========================================================" read -p "Press enter to
exit..."
```

Concepts Demonstrated

- Shell variables — storing values with VAR=value and using them with $VAR
- Command substitution — capturing command output into a variable with $(command)
- Built-in environment variables — $HOME is set by the shell automatically
- Conditional command availability check — command -v lsb_release before calling it
- Formatted output — using echo with string interpolation in double-quoted strings

Script 2 — FOSS Package Inspector (package_inspector.sh)

Description

This script detects which package manager is available on the running system and uses it to check whether python3 is installed. It prints the installed version if found. A case statement then prints a short philosophical note about each recognised open-source package.

Source Code

```
#!/bin/bash # Script 2: FOSS Package Inspector # Author: [YOUR NAME] | Reg: [YOUR REG NO]
# Course: Open Source Software | VIT Bhopal University PACKAGE="python3" # The software we
are auditing echo "FOSS Package Inspector" echo "----------------------" # if-elif-else:
detect the package manager and check installation accordingly if command -v dpkg
&>/dev/null; then # Debian-based (Ubuntu, Kali) — use dpkg if dpkg -l | grep -qw
"$PACKAGE"; then echo "$PACKAGE is installed (Debian-based system)." python3 --version
else echo "$PACKAGE is NOT installed." fi elif command -v rpm &>/dev/null; then #
RPM-based (Fedora, CentOS) — use rpm if rpm -q "$PACKAGE" &>/dev/null; then echo "$PACKAGE
is installed (RPM-based system)." rpm -qi "$PACKAGE" | grep -E 'Version|License|Summary'
else echo "$PACKAGE is NOT installed." fi else # Fallback: just check if the binary exists
in PATH if command -v python3 &>/dev/null; then echo "$PACKAGE found in PATH." python3
--version else echo "$PACKAGE is NOT installed." fi fi echo "" echo "Philosophy Notes"
echo "----------------" # case statement — prints a note based on which package is being
inspected case $PACKAGE in python3|python) echo "Python: born from frustration, shaped by
one idea —" echo " code should be readable by people, not just machines." echo " Licence:
PSF License — open and permissive." ;; git) echo "Git: Linus Torvalds built it in two
weeks because proprietary" echo " version control failed the Linux kernel project." ;;
apache2|httpd) echo "Apache: the open web server that proved free software could" echo "
power the internet better than any proprietary alternative." ;; vlc) echo "VLC: built by
students in Paris who needed to stream video" echo " across a university network.
Community-built; plays everything." ;; *) echo "$PACKAGE: an open-source tool contributing
to the FOSS ecosystem." ;; esac read -p "Press enter to exit..."
```

Concepts Demonstrated

- Nested if-elif-else blocks for package manager detection
- command -v to test whether a binary exists in $PATH before calling it
- case statement for pattern-matching on a variable value
- Piping — dpkg -l | grep -qw passes output from one command to another
- Output redirection — &>/dev/null discards both stdout and stderr

Script 3 — Disk and Permission Auditor (disk_auditor.sh)

Description

This script iterates over a predefined list of important Linux system directories using a for loop and reports the permissions, owner, group, and disk usage of each. It then additionally locates Python's system library directory and displays its permissions separately.

Source Code

```
#!/bin/bash # Script 3: Disk and Permission Auditor # Author: FAIZ KHAN | Reg: 24BCE10703
# Course: Open Source Software | VIT Bhopal University # Bash array — list of directories
to audit DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp") echo "Directory and Permission
Audit Report" echo "-------------------------------------" # for loop — iterate through
each item in the DIRS array for DIR in "${DIRS[@]}"; do # [ -d "$DIR" ] checks if the path
exists and is a directory if [ -d "$DIR" ]; then # ls -ld: list directory itself (not
contents) # awk '{print $1, $3, $4}' extracts: permissions, owner, group PERMS=$(ls -ld
"$DIR" | awk '{print $1, $3, $4}') # du -sh: disk usage in
human-readable form; 2>/dev/null suppresses errors SIZE=$(du -sh "$DIR" 2>/dev/null | cut
-f1) echo " $DIR" echo " Permissions/Owner : $PERMS" echo " Disk Usage : $SIZE" echo ""
else echo " $DIR — does not exist on this system" echo "" fi done echo "Python Library
Directory Check" echo "------------------------------" # find the Python3 library
directory under /usr/lib (depth 1 only) PYTHON_LIB=$(find /usr/lib -maxdepth 1 -name
"python3*" -type d 2>/dev/null | head -1) # -n "$PYTHON_LIB" checks that the variable is
non-empty (directory was found) if [ -n "$PYTHON_LIB" ]; then PERMS=$(ls -ld "$PYTHON_LIB"
| awk '{print $1, $3, $4}') SIZE=$(du -sh "$PYTHON_LIB" 2>/dev/null | cut -f1) echo "
Found: $PYTHON_LIB" echo " Permissions/Owner : $PERMS" echo " Disk Usage : $SIZE" else
echo " No Python3 library directory found under /usr/lib" fi # Check for the python3
binary itself if [ -f "/usr/bin/python3" ]; then echo "" echo " Python binary at
/usr/bin/python3:" ls -lh /usr/bin/python3 fi read -p "Press enter to exit..."
```

Concepts Demonstrated

- Bash arrays — declaring with ( ) and iterating with ${DIRS[@]}
- for loop — iterating over array elements
- Conditional test — [-d] for directory existence, [-f] for file, [-n] for non-empty string
- awk — extracting specific fields from ls -ld output by column number
- du -sh — human-readable disk usage; cut -f1 extracts the size field
- 2>/dev/null — redirecting stderr to suppress permission-denied errors

Script 4 — Log File Analyzer (log_analyzer.sh)

Description

This script accepts a log file path as its first command-line argument and an optional search keyword as the second (defaulting to error). It reads the file line by line using a while read loop, counts lines containing the keyword, prints a summary, and displays the last five matching lines.

Source Code

```
#!/bin/bash # Script 4: Log File Analyzer # Author: FAIZ KHAN | Reg: 24BCE10703 # Course: Open Source Software | VIT Bhopal University # Usage: ./log_analyzer.sh <logfile> [keyword] # $1 is the first command-line argument (the log file path) LOGFILE=$1 # ${2:-"error"} uses $2 if provided; otherwise defaults to "error" KEYWORD=${2:-"error"} COUNT=0 # counter variable — will be incremented for each matching line # Validate that an argument was actually passed if [ -z "$LOGFILE" ]; then echo "Usage: ./log_analyzer.sh <logfile> [keyword]" exit 1 fi # [ ! -f "$LOGFILE" ] checks that the file exists and is a regular file if [ ! -f "$LOGFILE" ]; then echo "Error: File '$LOGFILE' not found." exit 1 fi # [ ! -s "$LOGFILE" ] checks that the file is not empty if [ ! -s "$LOGFILE" ]; then echo "Warning: '$LOGFILE' is empty. Nothing to analyse." exit 0 fi echo "Log File Analyser" echo "-----------------" echo " File : $LOGFILE" echo " Keyword : '$KEYWORD'" echo "" # while IFS= read -r LINE: reads file line by line # IFS= prevents stripping of leading/trailing whitespace # -r prevents backslash interpretation while IFS= read -r LINE; do # grep -iq: case-insensitive (-i), quiet (-q: no output, just exit code) if echo "$LINE" | grep -iq "$KEYWORD"; then COUNT=$((COUNT + 1)) # arithmetic expansion: increment counter fi done < "$LOGFILE" # redirect file into the while loop as standard input echo " Result : Keyword '$KEYWORD' found $COUNT time(s)." echo "" echo "Last 5 Matching Lines:" echo "----------------------" # grep to extract matching lines, then tail to show only the last 5 MATCHES=$(grep -i "$KEYWORD" "$LOGFILE") if [ -z "$MATCHES" ]; then echo " No matching lines found." else echo "$MATCHES" | tail -5 fi read -p "Press enter to exit..."
```

Concepts Demonstrated

- Command-line arguments — $1, $2 for positional parameters
- Default parameter values — ${2:-"error"} syntax
- File test operators — [-z] (empty string), [! -f] (not a file), [! -s] (empty file)
- while IFS= read -r loop — standard idiom for line-by-line file processing
- Arithmetic expansion — $((COUNT + 1)) for integer arithmetic
- grep -i for case-insensitive search; tail -5 to limit to last five results

Script 5 — Open Source Manifesto Generator (manifesto.sh)

Description

This script interactively collects three responses from the user about their open-source beliefs and practices, then composes a personalised philosophy statement and saves it to a .txt file named after the current system user. The completed manifesto is displayed on screen using cat.

Source Code

```
#!/bin/bash # Script 5: Open Source Manifesto Generator # Author: FAIZ KHAN | Reg: 24BCE10703 # Course: Open Source Software | VIT Bhopal University # Alias note: developers often define aliases to simplify running scripts: # alias manifesto='bash ~/scripts/manifesto.sh' # This is one way the shell's alias feature connects to everyday open-source workflow. echo "Open Source Manifesto Generator" echo "================================" echo "" echo "Answer three questions to build your personal open-source statement." echo "" # read -p: display a prompt and wait for user input; store in variable read -p "1. Name one open-source tool you use every day: " TOOL read -p "2. In your own words, what does 'freedom' mean in software? " FREEDOM read -p "3. What is one thing you would build and share freely? " BUILD # date '+%d %B %Y': formats the date as "30 March 2026" DATE=$(date '+%d %B %Y') # whoami: returns the current Linux username; used to create a unique filename AUTHOR=$(whoami) # String concatenation in the filename — combines "manifesto_" with the username OUTPUT="manifesto_${AUTHOR}.txt" # > (redirect): creates the file (or overwrites if it exists) and writes the first line echo "On $DATE, I commit to the principles of open-source software." > "$OUTPUT" echo "" >> "$OUTPUT" # >> (append redirect): adds each subsequent line without overwriting echo "I believe that freedom in software means: $FREEDOM." >> "$OUTPUT" echo "Every day I rely on $TOOL — a tool built openly and shared freely by its community." >> "$OUTPUT" echo "Inspired by this, I intend to build $BUILD and release it for anyone to use," >> "$OUTPUT" echo "modify, and improve — because knowledge grows when it is shared." >> "$OUTPUT" echo "" >> "$OUTPUT" echo "Open source is not just a licence. It is a way of thinking:" >> "$OUTPUT" echo "that collaboration beats competition, and transparency beats secrecy." >> "$OUTPUT" echo "" >> "$OUTPUT" echo "Signed: $AUTHOR | $DATE" >> "$OUTPUT" echo "" echo "Manifesto saved to: $OUTPUT" echo "================================" echo "" cat "$OUTPUT" # display the saved file read -p "Press enter to exit..."
```

Concepts Demonstrated

- Interactive user input — read -p displays a prompt and stores the response
- Date formatting — date '+%d %B %Y' produces a human-readable date string
- Dynamic filename construction — manifesto_${AUTHOR}.txt creates a unique file per user
- File creation with > (overwrite) and appending with >>
- Displaying file contents with cat
- Alias concept — demonstrated in the comments as an example of shell convenience features

Conclusion

Python began as one developer's response to a frustration he experienced over a Christmas holiday. It became the world's most widely used programming language because of a decision that had nothing to do with the code itself: the decision to make it open. That single choice meant any developer anywhere could install it, any university could teach it, any company could build on it, and any contributor could improve it — all without asking for permission. The language's genuine technical virtues — its readability, its versatility, its enormous standard library — are real. But they are downstream of the open-source decision that made large-scale collaboration possible in the first place.

Auditing Python has clarified something concrete about what open-source means in practice. It is not an abstract philosophy. It is the specific binaries in /usr/bin, the specific packages in /usr/lib/python3, the specific governance decisions made through PEPs and Steering Council votes, and the specific licences that determine what companies and individuals are permitted to do with the result of other people's labour. Understanding each of those layers is part of understanding what it means to work in technology responsibly.

The five shell scripts in this project reinforced the connection between the philosophical and the practical. Open-source software is not abstract — it runs on a real system, under a real user, with real permissions, in real directories. The skills to inspect those systems — to read a log file, audit a directory's permissions, identify an installed package — are the skills that connect the principles of the open-source movement to the realities of operating a Linux environment. Learning both together is not a coincidence. It is the point.

References

[1] Van Rossum, G. (1991). Python 0.9.0 Release Notes. Centrum Wiskunde & Informatica, Amsterdam.

[2] Python Software Foundation. (2024). PSF License Agreement for Python 3.x. https://docs.python.org/3/license.html

[3] Python Software Foundation. (2024). History and License. https://docs.python.org/3/license.html#history-and-license

[4] GNU Project, Free Software Foundation. (2023). The Free Software Definition. https://gnu.org/philosophy/free-sw.html

[5] Open Source Initiative. (2023). The Open Source Definition (Version 1.9). https://opensource.org/osd

[6] Raymond, E. S. (1999). The Cathedral and the Bazaar. http://catb.org/~esr/writings/cathedral-bazaar

[7] Van Rossum, G., & Drake, F. L. (2009). Python 3 Reference Manual. Python Software Foundation.

[8] Shotts, W. E. (2019). The Linux Command Line (2nd ed.). No Starch Press. https://linuxcommand.org

[9] GNU Project. (2023). Bash Reference Manual. https://gnu.org/software/bash/manual

[10] SPDX License List. (2024). PSF-2.0. https://spdx.org/licenses/PSF-2.0.html

[11] MathWorks. (2024). MATLAB Pricing and Licensing. https://mathworks.com/pricing-licensing.html

[12] PyPI. (2024). PyPI Statistics. https://pypi.org/stats/

[13] Python Steering Council. (2024). Python Enhancement Proposals Index. https://peps.python.org

[14] Heartbleed. (2014). CVE-2014-0160. https://heartbleed.com